Αναγνώριση Προτύπων - Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 13, 2025

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Κατόπιν, υλοποιούμε τη συνάρτηση *parse_free_digits* προκειμένου να διαβάσουμε σωστά τα αρχεία με τη βιβλιοθήκη librosa.

```
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Το αποτέλεσμα της κλήσης της συνάρτησης είναι:

Total wavs: 3000. Fs = 8000 Hz

Στην συνέχεια βασισμένοι στην υπολοποίηση του βοηθητικού κώδικα ορίζουμε τη δικιά μας συνάρτηση *split_free_digits* την οποία παρουσιάζουμε παρακάτω:

```
def extract_features(wavs, n_mfcc=6, Fs=8000):
    # Extract MFCCs for all wavs
    window = 30 * Fs // 1000
    step = window // 2
    frames = [
        librosa.feature.mfcc(
            y=wav, sr=Fs, n_fft=window, hop_length=window - step, n_mfcc=n_mfcc
        ).T
        for wav in tqdm(wavs, desc="Extracting mfcc features...")
```

```
    ]

    print("Feature extraction completed with {} mfccs per frame".format(n_mfcc)
    )

    return frames


def split_free_digits(frames, ids, speakers, labels):
    print("Splitting in train test split using the default dataset split")
    # Split to train-test
    X_train, y_train, spk_train = [], [], []
    X_test, y_test, spk_test = [], [], []
    test_indices = ["0", "1", "2", "3", "4"]

    for idx, frame, label, spk in zip(ids, frames, labels, speakers):
        if str(idx) in test_indices:
            X_test.append(frame)
            y_test.append(label)
            spk_test.append(spk)
        else:
            X_train.append(frame)
            y_train.append(label)
            spk_train.append(spk)

    return X_train, X_test, y_train, y_test, spk_train, spk_test
```

Τρέχουμε τον κώδικα για να επιβεβαιώσουμε την ορθή λειτουργία του:

```
└─ python3 lab.py
Total wavs: 3000. Fs = 8000 Hz
Splitting in train test split using the default dataset split
Train dataset's size is 2400
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 9
Train dataset has 60 samples of digit 9
```

Βλέπουμε ότι το split που έχουμε κάνει έχει μέγεθος 2400, δηλαδή είναι πράγματι το 80% του 3000. Επιπλέον, είναι ισορροπημένο, αφού υπάρχει ίδιος αριθμός δειγμάτων ήχου σε κάθε κατηγορία - ψηφίο (60). Επομένως ως μετρική του *accuracy* θα είναι αρκετό για να αξιολογήσουμε την επίδοση των ταξινομητών μας (όπως είδαμε και στην προπαρασκευή, στην περίπτωση του ισορροπημένου dataset το F1-score ακολουθεί το ίδιο trend με το accuracy).

Αφού επιβεβαιώσαμε την ορθή λειτουργία της ρουτίνας μας αφαιρούμε - μετατρέπουμε σε σχόλιο τον κώδικα για το debugging.

Βήμα 10

Βήμα 11

Βήμα 12

Βήμα 13

Βήμα 14